

Python OOP Interview Questions — Top 100

Classes · Inheritance · Dunder Methods

SECTION 1 — Classes & Instances (Q1–Q25)

Q1. What is a class in Python? How do you define one?

A class is a blueprint for creating objects. It bundles data (attributes) and behaviour (methods) together.

```
class Animal:
    species = "Unknown"          # class attribute

    def __init__(self, name):
        self.name = name         # instance attribute

    def speak(self):
        return f"{self.name} makes a sound"

dog = Animal("Rex")
print(dog.speak())              # Rex makes a sound
```

Q2. What is the difference between a class attribute and an instance attribute?

	Class attribute	Instance attribute
Defined	In class body	Inside <code>__init__</code> (or other methods)
Shared	All instances share one copy	Each instance has its own copy
Access	<code>Class.attr</code> or <code>self.attr</code>	<code>self.attr</code> only

```
class Counter:
    count = 0                    # class attribute

    def __init__(self):
        Counter.count += 1
        self.id = Counter.count # instance attribute

a, b = Counter(), Counter()
```

```
print(Counter.count)    # 2
print(a.id, b.id)       # 1 2
```

Q3. What does `self` refer to?

`self` is a reference to the **current instance**. Python passes it automatically as the first argument to every instance method. The name `self` is a convention, not a keyword.

```
class Point:
    def __init__(self, x, y):
        self.x = x    # self == the object being created
        self.y = y

    def distance(self):
        return (self.x**2 + self.y**2) ** 0.5

p = Point(3, 4)
print(p.distance())   # 5.0
```

Q4. What is `__init__`? Is it a constructor?

`__init__` is an **initializer**, not the true constructor. The constructor is `__new__`, which allocates the object. `__init__` then initializes it.

```
class Foo:
    def __new__(cls, *args, **kwargs):
        print("__new__ called")
        return super().__new__(cls)

    def __init__(self, value):
        print("__init__ called")
        self.value = value

f = Foo(42)
# __new__ called
# __init__ called
```

Q5. What are class methods and static methods? How do they differ from instance methods?

```
class MyClass:
    count = 0

    def instance_method(self):    # receives instance
```

```

        return self

    @classmethod
    def class_method(cls):
        # receives the class
        cls.count += 1
        return cls.count

    @staticmethod
    def static_method(x, y):
        # receives nothing special
        return x + y

```

- **Instance method:** operates on `self`; can access/modify instance *and* class state.
- **Class method:** operates on `cls`; commonly used as factory constructors.
- **Static method:** no implicit first arg; utility function logically grouped with the class.

Q6. How do you create a class with a private attribute in Python?

Python uses **name mangling** for double-underscore prefixes (`__attr` → `_ClassName_attr`). Single underscore is a *convention* meaning “internal use”.

```

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # name-mangled

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

acc = BankAccount(100)
acc.deposit(50)
print(acc.get_balance())           # 150
# print(acc.__balance)             # AttributeError
print(acc._BankAccount__balance)   # 150 - accessible but discouraged

```

Q7. What is a property? How do you use @property?

@property lets you define getter/setter/deleter logic behind a simple attribute interface.

```

class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius

```

```

@property
def celsius(self):
    return self._celsius

@celsius.setter
def celsius(self, value):
    if value < -273.15:
        raise ValueError("Below absolute zero!")
    self._celsius = value

@property
def fahrenheit(self):
    return self._celsius * 9/5 + 32

t = Temperature(25)
print(t.fahrenheit)    # 77.0
t.celsius = 100
print(t.fahrenheit)    # 212.0

```

Q8. What is `__slots__`? When should you use it?

`__slots__` restricts the attributes an instance can have, eliminating the per-instance `__dict__` and reducing memory.

```

class Point:
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(1, 2)
# p.z = 3    # AttributeError - z not in __slots__

```

Use it when creating **millions of small objects** and memory is a bottleneck. Downside: can't add arbitrary attributes or use `__dict__`-based features like `vars()`.

Q9. How does Python resolve attribute lookup (MRO)?

Python first checks the **instance**, then the **class**, then **base classes** in MRO order (C3 linearisation).

```

class A:
    x = "A"

```

```

class B(A):
    pass

b = B()
b.x = "instance"
print(b.x)      # instance (instance wins)
del b.x
print(b.x)      # A (class via MRO)

```

Q10. What is the difference between `isinstance()` and `type()`?

```

class Animal: pass
class Dog(Animal): pass

d = Dog()
print(type(d) == Dog)      # True
print(type(d) == Animal)   # False + doesn't consider inheritance
print(isinstance(d, Dog))  # True
print(isinstance(d, Animal)) # True + respects inheritance

```

Prefer `isinstance()` for type checks in production code.

Q11. Explain the `@dataclass` decorator.

`dataclass` auto-generates `__init__`, `__repr__`, `__eq__` (and optionally `__hash__`, `__lt__`, etc.).

```

from dataclasses import dataclass, field

@dataclass(order=True)
class Point:
    x: float
    y: float
    label: str = field(default="", compare=False)

p1 = Point(1.0, 2.0)
p2 = Point(1.0, 2.0, "origin")
print(p1 == p2)      # True (label excluded from compare)
print(p1 < Point(2.0, 0.0)) # True

```

Q12. What is the difference between `__class__` and `type(obj)`?

In almost all cases they are identical. They diverge only if `__class__` is overridden (possible with metaclasses or `__class__` assignment in some edge cases).

Prefer `type(obj)` for reflection; `__class__` for descriptors and `super()` machinery.

Q13. What is a mixin and when do you use it?

A mixin is a small class that provides a specific set of methods meant to be mixed into other classes via multiple inheritance. It should not be instantiated on its own.

```
class JsonMixin:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)

class LogMixin:
    def log(self):
        print(f"[LOG] {self!r}")

class User(JsonMixin, LogMixin):
    def __init__(self, name, age):
        self.name = name
        self.age = age

u = User("Alice", 30)
print(u.to_json()) # {"name": "Alice", "age": 30}
u.log()
```

Q14. How do you implement a Singleton in Python?

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

a = Singleton()
b = Singleton()
print(a is b) # True
```

Or using a module-level variable (idiomatic Python).

Q15. What is `__dict__` on a class vs an instance?

```

class Dog:
    breed = "Labrador"
    def __init__(self, name):
        self.name = name

d = Dog("Rex")
print(d.__dict__)      # {'name': 'Rex'}           - instance attrs only
print(Dog.__dict__)    # mappingproxy with breed, __init__, etc.

```

Q16. Can you delete a class attribute at runtime?

```

class Config:
    debug = True

del Config.debug
# print(Config.debug) # AttributeError

```

Yes. You can also use `delattr(Config, "debug")`.

Q17. What is `__init_subclass__`?

Called on the **parent** class whenever it is subclassed. Useful for plugin registration without metaclasses.

```

class Plugin:
    _registry = {}

    def __init_subclass__(cls, name, **kwargs):
        super().__init_subclass__(**kwargs)
        Plugin._registry[name] = cls

class AudioPlugin(Plugin, name="audio"):
    pass

print(Plugin._registry)  # {'audio': <class 'AudioPlugin'>}

```

Q18. What is `__class_getitem__`?

Allows a class to support generic notation like `MyClass[int]`.

```

class Stack:
    def __class_getitem__(cls, item):
        return f"Stack[{item.__name__}]"

print(Stack[int])      # Stack[int]

```

This is how `list[int]`, `dict[str, int]` work in Python 3.9+.

Q19. Difference between `object.__setattr__` and normal attribute assignment?

Inside `__setattr__` or when you need to bypass descriptors, call `object.__setattr__(self, name, value)` directly to avoid infinite recursion.

```
class Guarded:
    def __setattr__(self, name, value):
        if name.startswith("_"):
            raise AttributeError("Private!")
        object.__setattr__(self, name, value)  # bypass our own hook

g = Guarded()
g.x = 10  # OK
# g._x = 10  # AttributeError
```

Q20. What is a descriptor?

An object that customizes attribute access via `__get__`, `__set__`, `__delete__`.

```
class Positive:
    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        return obj.__dict__.get(self.name, 0)

    def __set__(self, obj, value):
        if value < 0:
            raise ValueError(f"{self.name} must be positive")
        obj.__dict__[self.name] = value

class Rectangle:
    width = Positive()
    height = Positive()

r = Rectangle()
r.width = 5
r.height = 3
# r.width = -1  # ValueError
```


Q21. What are `__get__`, `__set__`, `__delete__` in descriptors?

- `__get__(self, obj, type)` — attribute read
- `__set__(self, obj, value)` — attribute write
- `__delete__(self, obj)` — `del obj.attr`

A **data descriptor** implements at least `__set__` or `__delete__`; it has priority over instance `__dict__`. A **non-data descriptor** only has `__get__`; instance `dict` takes priority.

Q22. How are `classmethod` and `staticmethod` implemented internally?

Both are **non-data descriptors**. `classmethod.__get__` returns a bound method with `cls` as first arg. `staticmethod.__get__` returns the raw function unchanged.

Q23. What is `__call__`?

Makes an instance callable like a function.

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, x):
        return x * self.factor
```

```
double = Multiplier(2)
print(double(7))    # 14
```

Q24. What is the purpose of `__new__`?

`__new__` allocates and returns the new instance. Override it to control object creation (e.g., Singletons, immutable subclasses of built-ins).

```
class PositiveInt(int):
    def __new__(cls, value):
        if value <= 0:
            raise ValueError("Must be positive")
        return super().__new__(cls, value)
```

```
x = PositiveInt(5)
# PositiveInt(-1) # ValueError
```

Q25. How do you make a class immutable?

```

class Immutable:
    def __init__(self, x, y):
        object.__setattr__(self, "x", x)
        object.__setattr__(self, "y", y)

    def __setattr__(self, name, value):
        raise AttributeError("Immutable!")

    def __delattr__(self, name):
        raise AttributeError("Immutable!")

p = Immutable(1, 2)
# p.x = 10 # AttributeError

```

SECTION 2 — Inheritance (Q26–Q55)

Q26. What is inheritance in Python?

Inheritance lets a child class reuse and extend the behaviour of a parent class.

```

class Shape:
    def area(self):
        raise NotImplementedError

class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        import math
        return math.pi * self.r ** 2

c = Circle(5)
print(c.area()) # 78.53...

```

Q27. What is multiple inheritance? What problem does it introduce?

A class can inherit from more than one base. It introduces the **Diamond Problem** — ambiguity about which parent's method to call.

```

class A:
    def greet(self): return "A"

class B(A):

```

```

    def greet(self): return "B"

class C(A):
    def greet(self): return "C"

class D(B, C):
    pass

print(D().greet())      # B - MRO decides
print(D.__mro__)        # D → B → C → A → object

```

Q28. Explain C3 Linearisation (MRO).

Python uses the **C3 linearisation** algorithm to determine the Method Resolution Order. The rule: a class always appears before its parents, and parents appear in the order listed.

```

class X: pass
class Y: pass
class Z(X, Y): pass
print(Z.__mro__)      # Z → X → Y → object

```

Q29. What does super() do?

super() returns a proxy that delegates method calls to the **next class in the MRO**, enabling cooperative multiple inheritance.

```

class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)    # calls Animal.__init__
        self.breed = breed

d = Dog("Rex", "Labrador")
print(d.name, d.breed)

```

Q30. Why is cooperative super() important in multiple inheritance?

```

class A:
    def method(self):
        print("A")

```

```

class B(A):
    def method(self):
        super().method()    # passes control to C, not A directly
        print("B")

class C(A):
    def method(self):
        super().method()
        print("C")

class D(B, C):
    def method(self):
        super().method()
        print("D")

D().method()
# A  C  B  D    (MRO order, bottom-up)

```

Q31. What is method overriding?

A child class provides its own implementation of a method already defined in the parent.

```

class Vehicle:
    def start(self):
        return "Starting vehicle"

class Car(Vehicle):
    def start(self):
        return "Starting car engine"    # overrides

c = Car()
print(c.start())    # Starting car engine

```

Q32. What is method overloading in Python?

Python does **not** natively support overloading (same name, different signatures). You simulate it with default args, `*args`, or `singledispatch`.

```

from functools import singledispatch

@singledispatch
def process(arg):
    raise NotImplementedError

@process.register(int)

```

```
def _(arg):
    return f"int: {arg * 2}"

@process.register(str)
def _(arg):
    return f"str: {arg.upper()}"

print(process(5))      # int: 10
print(process("hi"))   # str: HI
```

Q33. How do you call a parent class method explicitly?

```
class Parent:
    def greet(self):
        return "Hello from Parent"

class Child(Parent):
    def greet(self):
        parent_msg = super().greet()
        return f"{parent_msg} and Child"

print(Child().greet())
```

Q34. What is `__bases__` and `__mro__`?

```
class A: pass
class B(A): pass

print(B.__bases__)  # (<class 'A'>,) - direct parents only
print(B.__mro__)    # (<class 'B'>, <class 'A'>, <class 'object'>)
```

Q35. What is abstract base class (ABC)?

An ABC defines a contract that subclasses must fulfil. Trying to instantiate an ABC or a subclass that doesn't implement all abstract methods raises `TypeError`.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

    @abstractmethod
    def perimeter(self) -> float: ...
```

```

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

    def perimeter(self):
        return 4 * self.side

# Shape()      # TypeError
s = Square(4)
print(s.area())    # 16

```

Q36. Can you use `@abstractmethod` with `@classmethod` or `@staticmethod`?

```

from abc import ABC, abstractmethod

```

```

class Base(ABC):
    @classmethod
    @abstractmethod
    def create(cls): ...

    @staticmethod
    @abstractmethod
    def validate(value): ...

```

Yes. Put `@abstractmethod` as the **innermost** decorator.

Q37. What is `__subclasses__()`?

Returns a list of immediate (direct) subclasses.

```

class Plugin: pass
class AudioPlugin(Plugin): pass
class VideoPlugin(Plugin): pass

print(Plugin.__subclasses__())
# [<class 'AudioPlugin'>, <class 'VideoPlugin'>]

```

Q38. What happens when a subclass does not call `super().__init__()`?

The parent's `__init__` is skipped, so parent attributes are never set — causing `AttributeError` if you try to access them.

```

class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, breed):
        self.breed = breed  # forgot super().__init__()

d = Dog("Lab")
# print(d.name)  # AttributeError

```

Q39. What is the difference between composition and inheritance?

- **Inheritance** (“is-a”): Dog is an Animal.
- **Composition** (“has-a”): Car has an Engine.

Composition is often preferred (more flexible, avoids fragile base class problem).

```

class Engine:
    def start(self): return "Vroom"

class Car:
    # composition, not inheritance
    def __init__(self):
        self.engine = Engine()

    def drive(self):
        return self.engine.start()

```

Q40. How do you prevent a class from being subclassed?

Override `__init_subclass__` to raise an error, or use a metaclass.

```

class Final:
    def __init_subclass__(cls, **kwargs):
        raise TypeError("Cannot subclass Final")

# class Sub(Final): pass  # TypeError

```

Q41. What is duck typing?

“If it walks like a duck and quacks like a duck, it’s a duck.” Python checks behaviour (methods present), not type hierarchy.

```

class Duck:
    def quack(self): return "Quack!"

```

```

class Person:
    def quack(self): return "I'm quacking like a duck!"

def make_it_quack(duck):
    print(duck.quack())    # works for any object with .quack()

make_it_quack(Duck())
make_it_quack(Person())

```

Q42. What is `__subclasshook__`?

A classmethod on an ABC that lets you customise `isinstance`/`issubclass` checks without requiring explicit subclassing.

```

from abc import ABC, abstractmethod

class Drawable(ABC):
    @abstractmethod
    def draw(self): ...

    @classmethod
    def __subclasshook__(cls, C):
        if cls is Drawable:
            if any("draw" in B.__dict__ for B in C.__mro__):
                return True
            return NotImplemented

class Sprite:
    def draw(self): print("drawing sprite")

print(isinstance(Sprite(), Drawable))    # True - no explicit inheritance!

```

Q43. How do you use `super()` in Python 2 vs Python 3?

Python 2: `super(ClassName, self).method()` — must pass class and instance.

Python 3: `super().method()` — zero-argument form works via `__class__` cell.

Q44. What is the fragile base class problem?

When a base class changes its internal implementation, subclasses that depend on those internals break — even without modifying the subclass.

Solution: document what subclasses are allowed to override; use composition where possible; avoid calling overridable methods from `__init__`.

Q45. Explain cooperative multiple inheritance with `**kwargs`.

```
class A:
    def __init__(self, *, a=0, **kwargs):
        super().__init__(**kwargs)
        self.a = a

class B:
    def __init__(self, *, b=0, **kwargs):
        super().__init__(**kwargs)
        self.b = b

class C(A, B):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)

c = C(a=1, b=2)
print(c.a, c.b)    # 1 2
```

Each class strips its own keyword args and passes the rest up the chain.

Q46. What is `issubclass()`?

```
class Animal: pass
class Dog(Animal): pass

print(issubclass(Dog, Animal))    # True
print(issubclass(Animal, Dog))    # False
print(issubclass(Dog, object))    # True - everything inherits object
```

Q47. How does Python handle diamond inheritance with `super()`?

The MRO guarantees each class in the diamond is called exactly once, in C3 order, as long as all classes use `super()` cooperatively.

Q48. What is a virtual subclass?

Registered with `ABC.register()` — treated as a subclass for `isinstance/issubclass` without actual inheritance.

```
from abc import ABC

class Sequence(ABC): pass
```

```
Sequence.register(list)
print(isinstance([], Sequence))    # True
print(issubclass(list, Sequence)) # True
```

Q49. What is `__init_subclass__` vs metaclass for class customisation?

`__init_subclass__` is simpler and covers most use cases (registration, enforcement, defaults). Metaclasses are required for more invasive changes: modifying the class *creation* process itself (custom `__new__`/`__init__` on the metaclass, custom `__prepare__`).

Q50. How do you call a sibling class method in multiple inheritance?

You don't call it directly — that's the job of `super()` and the MRO. Trust cooperative inheritance to invoke each class in the correct order.

Q51. What is method resolution with diamond and non-cooperative classes?

If a base class doesn't call `super()`, the chain breaks at that point and sibling classes are skipped.

Q52. What is the difference between `__base__` and `__bases__`?

```
class A: pass
class B(A): pass
class C(A, B): pass    # error actually - invalid MRO; simpler:

class D(A): pass
print(D.__base__)      # <class 'A'> - first base (single inheritance shortcut)
print(D.__bases__)     # (<class 'A'>,) - tuple of all direct bases
```

Q53. How do you override `__new__` correctly in a subclass?

Always call `super().__new__(cls)` and return the result. Don't pass extra args to `object.__new__` when `__init__` is defined.

```
class TrackedList(list):
    def __new__(cls, *args, **kwargs):
        instance = super().__new__(cls)
        instance.creation_count = 0
        return instance
```

```
tl = TrackedList([1, 2, 3])
print(tl, tl.creation_count) # [1, 2, 3] 0
```

Q54. Can a class inherit from a built-in type?

Yes.

```
class MyList(list):
    def sum(self):
        return sum(self)

m1 = MyList([1, 2, 3])
print(m1.sum()) # 6
print(m1[0]) # 1 - list behaviour inherited
```

Q55. What is `super()` without arguments equivalent to?

`super()` inside a method is equivalent to `super(__class__, self)` where `__class__` is a closure cell set by the compiler — it always refers to the class the method was *defined in*, not the instance’s actual class.

SECTION 3 — Dunder (Magic) Methods (Q56–Q100)

Q56. What are dunder methods?

Special methods with double-underscore prefix and suffix (`__method__`) that Python calls implicitly when certain syntax or built-in operations are used. Also called “magic methods” or “special methods”.

Q57. `__str__` vs `__repr__` — what’s the difference?

	<code>__repr__</code>	<code>__str__</code>
Goal	Unambiguous developer representation	Readable user-facing string
Fallback	Used when <code>__str__</code> missing	Falls back to <code>__repr__</code>
Called by	<code>repr(obj)</code> , interactive REPL	<code>str(obj)</code> , <code>print(obj)</code>

```
class Point:
    def __init__(self, x, y):
```

```

        self.x, self.y = x, y

    def __repr__(self):
        return f"Point({self.x!r}, {self.y!r})"

    def __str__(self):
        return f"({self.x}, {self.y})"

p = Point(1, 2)
print(repr(p))    # Point(1, 2)
print(str(p))    # (1, 2)
print(p)         # (1, 2)

```

Q58. How do you implement arithmetic operators?

```

class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Vector(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar):
        return Vector(self.x * scalar, self.y * scalar)

    def __rmul__(self, scalar):    # 3 * v
        return self.__mul__(scalar)

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)    # Vector(4, 6)
print(3 * v1)    # Vector(3, 6)

```

Q59. What are reflected (right-hand) operators like `__radd__`?

When `a + b` is called and `a.__add__(b)` returns `NotImplemented`, Python tries `b.__radd__(a)`.

```

class Money:

```

```

def __init__(self, amount):
    self.amount = amount

def __add__(self, other):
    if isinstance(other, (int, float)):
        return Money(self.amount + other)
    return NotImplemented

def __radd__(self, other):    # allows: 10 + Money(5)
    return self.__add__(other)

print((Money(5) + 3).amount)    # 8
print((10 + Money(5)).amount)  # 15

```

Q60. How do you implement comparison operators?

```

from functools import total_ordering

@total_ordering
class Student:
    def __init__(self, gpa):
        self.gpa = gpa

    def __eq__(self, other):
        return self.gpa == other.gpa

    def __lt__(self, other):
        return self.gpa < other.gpa

# With @total_ordering, __le__, __gt__, __ge__ are auto-derived
s1, s2 = Student(3.5), Student(3.8)
print(s1 < s2)    # True
print(s1 >= s2)   # False

```

Q61. What is `__hash__` and when must you define it?

If you define `__eq__`, Python sets `__hash__ = None` (making the object unhashable). You must also define `__hash__` if you want the object in sets/dicts.

```

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)

```

```

def __hash__(self):
    return hash((self.x, self.y))

s = {Point(1, 2), Point(1, 2)}
print(len(s))    # 1

```

Q62. Implement a container using `__len__`, `__getitem__`, `__setitem__`, `__delitem__`.

```

class Matrix:
    def __init__(self, rows, cols):
        self._data = [[0]*cols for _ in range(rows)]
        self.rows, self.cols = rows, cols

    def __len__(self):
        return self.rows * self.cols

    def __getitem__(self, key):
        r, c = key
        return self._data[r][c]

    def __setitem__(self, key, value):
        r, c = key
        self._data[r][c] = value

    def __delitem__(self, key):
        r, c = key
        self._data[r][c] = 0

m = Matrix(2, 2)
m[0, 1] = 7
print(m[0, 1])    # 7
print(len(m))     # 4

```

Q63. How do you make an object iterable (`__iter__`, `__next__`)?

```

class Countdown:
    def __init__(self, n):
        self.n = n

    def __iter__(self):
        self.current = self.n
        return self

```

```

def __next__(self):
    if self.current <= 0:
        raise StopIteration
    val = self.current
    self.current -= 1
    return val

for x in Countdown(3):
    print(x)    # 3 2 1

```

Q64. What is the iterator protocol vs the iterable protocol?

- **Iterable:** has `__iter__()` returning an iterator.
- **Iterator:** has both `__iter__()` (returns `self`) and `__next__()`.

All iterators are iterables, but not vice versa (e.g., a list is iterable but not an iterator).

Q65. How does `__contains__` work?

Called for `in` operator. If not defined, Python falls back to iterating and comparing.

```

class EvenNumbers:
    def __contains__(self, item):
        return isinstance(item, int) and item % 2 == 0

evens = EvenNumbers()
print(4 in evens)    # True
print(3 in evens)    # False

```

Q66. Implement `__enter__` and `__exit__` (context manager).

```

class ManagedFile:
    def __init__(self, path, mode="r"):
        self.path = path
        self.mode = mode

    def __enter__(self):
        self.file = open(self.path, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        return False    # don't suppress exceptions

```

```
with ManagedFile("/tmp/test.txt", "w") as f:
    f.write("hello")
```

Q67. What does `__exit__` returning `True` mean?

It suppresses the exception that occurred inside the `with` block. Returning `False` or `None` re-raises it.

Q68. What is `__getattr__` vs `__getattribute__`?

- `__getattribute__` — called on **every** attribute access (override carefully).
- `__getattr__` — called only when normal lookup **fails** (safe to override for fallback).

```
class FlexObj:
    def __getattr__(self, name):
        return f"<missing: {name}>"
```

```
f = FlexObj()
print(f.foo)    # <missing: foo>
```

Q69. How do you implement `__setattr__` and `__delattr__`?

```
class ReadOnly:
    def __init__(self, **kwargs):
        for k, v in kwargs.items():
            object.__setattr__(self, k, v)

    def __setattr__(self, name, value):
        raise AttributeError("Read-only object")

    def __delattr__(self, name):
        raise AttributeError("Read-only object")
```

```
r = ReadOnly(x=1, y=2)
print(r.x)    # 1
# r.x = 10    # AttributeError
```

Q70. What is `__missing__` in a dict subclass?

Called when a key is not found in `__getitem__`.


```

class DefaultDict(dict):
    def __missing__(self, key):
        self[key] = 0
        return 0

d = DefaultDict()
d["a"] += 5
print(d)    # {'a': 5}

```

Q71. How do you implement `__format__`?

Called by `format()` and f-string format specs.

```

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __format__(self, spec):
        if spec == "polar":
            import math
            r = math.hypot(self.x, self.y)
            theta = math.atan2(self.y, self.x)
            return f"({r:.2f}, {theta:.2f}rad)"
        return f"({self.x}, {self.y})"

p = Point(3, 4)
print(f"{p}")           # (3, 4)
print(f"{p:polar}")     # (5.00, 0.93rad)

```

Q72. What does `__bool__` do?

Called by `bool()`, `if`, `while`. If absent, Python falls back to `__len__`.

```

class Bag:
    def __init__(self, items):
        self.items = items

    def __bool__(self):
        return len(self.items) > 0

    def __len__(self):
        return len(self.items)

b = Bag([])
print(bool(b))    # False

```

```
if not b:
    print("Bag is empty")
```

Q73. What are in-place operators (`__iadd__`, `__imul__`, etc.)?

Called for `+=`, `*=`, etc. Should mutate `self` and return `self`.

```
class Stack:
    def __init__(self):
        self._data = []

    def __iadd__(self, item):
        self._data.append(item)
        return self

    def __repr__(self):
        return f"Stack({self._data})"
```

```
s = Stack()
s += 1
s += 2
print(s)    # Stack([1, 2])
```

Q74. What is `__index__`?

Called when an integer index is needed (`slice`, `bin()`, `hex()`, `oct()`).

```
class Ordinal:
    def __init__(self, n):
        self.n = n

    def __index__(self):
        return self.n

lst = [10, 20, 30]
print(lst[Ordinal(1)])    # 20
print(bin(Ordinal(10)))   # 0b1010
```

Q75. What is `__len__`?

Called by `len()`. Must return a non-negative integer.

Q76. Explain `__slots__` interaction with inheritance.

If a subclass doesn't define `__slots__`, it gets a `__dict__` anyway.

```

class A:
    __slots__ = ("x",)

class B(A):
    pass    # B has __dict__ despite A's slots

b = B()
b.x = 1
b.z = 99    # works - B has __dict__

```

Q77. What is `__sizeof__`?

Returns the size of the object in bytes (not recursively). Used by `sys.getsizeof()`.

Q78. Implement `__copy__` and `__deepcopy__`.

```

import copy

class Graph:
    def __init__(self, nodes):
        self.nodes = nodes

    def __copy__(self):
        return Graph(self.nodes)    # shallow: shared list

    def __deepcopy__(self, memo):
        return Graph(copy.deepcopy(self.nodes, memo))    # deep copy

g1 = Graph([1, 2, 3])
g2 = copy.copy(g1)
g3 = copy.deepcopy(g1)

```

Q79. What is `__reduce__` and `__reduce_ex__`?

Called by pickle to determine how to serialize an object. `__reduce_ex__` takes a protocol version.

```

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __reduce__(self):
        return (Point, (self.x, self.y))

```

```
import pickle
p = Point(1, 2)
data = pickle.dumps(p)
p2 = pickle.loads(data)
print(p2.x, p2.y)    # 1 2
```

Q80. What are `__getstate__` and `__setstate__`?

Used by pickle to customise serialization/deserialization without overriding `__reduce__`.

```
class Config:
    def __init__(self, host, port, secret):
        self.host = host
        self.port = port
        self.secret = secret

    def __getstate__(self):
        state = self.__dict__.copy()
        del state["secret"]    # don't pickle sensitive data
        return state

    def __setstate__(self, state):
        self.__dict__.update(state)
        self.secret = None    # reset after unpickling
```

Q81. What is `__del__`? Is it a destructor?

`__del__` is a **finalizer**, called when the object's reference count drops to zero. It's not guaranteed to be called promptly (or at all in CPython with cycles). Don't rely on it for resource cleanup — use context managers.

```
class Resource:
    def __del__(self):
        print("Resource cleaned up")
```

Q82. What is `__class_getitem__`? How does it enable generics?

```
class TypedList:
    def __class_getitem__(cls, item):
        # Returns a generic alias
        return f"{cls.__name__}[{item.__name__}]"

print(TypedList[int])    # TypedList[int]
```

In typing and `collections.abc`, this powers `list[int]`, `dict[str, int]` syntax.

Q83. What is `__init_subclass__` useful for?

Automatic subclass registration, enforcing interface contracts, setting class-level defaults.

```
class Validator:
    _validators = {}

    def __init_subclass__(cls, field, **kwargs):
        super().__init_subclass__(**kwargs)
        Validator._validators[field] = cls

class EmailValidator(Validator, field="email"):
    pass

print(Validator._validators)  # {'email': <class 'EmailValidator'>}
```

Q84. What is `__set_name__`?

Called when a descriptor is assigned inside a class body. Lets the descriptor know its own attribute name.

```
class Typed:
    def __set_name__(self, owner, name):
        self.name = name
        self.storage_name = f"_{name}"

    def __get__(self, obj, objtype=None):
        if obj is None: return self
        return getattr(obj, self.storage_name, None)

    def __set__(self, obj, value):
        setattr(obj, self.storage_name, value)

class Person:
    name = Typed()
    age = Typed()

p = Person()
p.name = "Alice"
print(p.name)  # Alice
```

Q85. What is `__prepare__`?

A metaclass hook called before the class body is executed. Returns the namespace (dict) used during class creation.

```
class OrderedMeta(type):
    @classmethod
    def __prepare__(mcs, name, bases):
        return {} # or an OrderedDict to preserve definition order

class MyClass(metaclass=OrderedMeta):
    x = 1
    y = 2
```

Q86. What is `__instancecheck__` and `__subclasscheck__`?

Metaclass hooks that customise `isinstance()` and `issubclass()`.

```
class Meta(type):
    def __instancecheck__(cls, instance):
        return hasattr(instance, "quack")

class Ducklike(metaclass=Meta):
    pass

class FakeDuck:
    def quack(self): pass

print(isinstance(FakeDuck(), Ducklike)) # True
```

Q87. What is `__abs__`, `__neg__`, `__pos__`, `__invert__`?

Unary operators: `abs()`, `-obj`, `+obj`, `~obj`.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __abs__(self):
        return (self.x**2 + self.y**2) ** 0.5

    def __neg__(self):
        return Vector(-self.x, -self.y)

v = Vector(3, 4)
print(abs(v)) # 5.0
print(-v) # Vector(-3, -4) - needs __repr__ too
```

Q88. What is `__round__`, `__floor__`, `__ceil__`, `__trunc__`?

Support `round()`, `math.floor()`, `math.ceil()`, `math.trunc()`.

```
import math

class Money:
    def __init__(self, amount):
        self.amount = amount

    def __round__(self, ndigits=0):
        return Money(round(self.amount, ndigits))

    def __floor__(self):
        return Money(math.floor(self.amount))

m = Money(3.75)
print(round(m, 1).amount)      # 3.8
print(math.floor(m).amount)    # 3
```

Q89. What is `__matmul__` (@ operator)?

Added in Python 3.5 for matrix multiplication (PEP 465). Used by NumPy arrays.

```
class Matrix:
    def __init__(self, data):
        self.data = data

    def __matmul__(self, other):
        # simple 2x2 demo
        a, b = self.data, other.data
        return Matrix([[
            sum(a[i][k]*b[k][j] for k in range(len(b)))
            for j in range(len(b[0]))]
            for i in range(len(a))])
```

Q90. What does `__divmod__` do?

Called by `divmod(a, b)`.

```
class Duration:
    def __init__(self, seconds):
        self.seconds = seconds
```

```
def __divmod__(self, other):
    q, r = divmod(self.seconds, other.seconds)
    return Duration(q), Duration(r)
```

Q91. What is `__fspath__`?

Called by `os.fspath()`. Lets objects act as file system paths.

```
class MyPath:
    def __init__(self, path):
        self._path = path

    def __fspath__(self):
        return self._path

import os
p = MyPath("/tmp/file.txt")
print(os.fspath(p))    # /tmp/file.txt
```

Q92. What is `__await__`?

Makes an object usable with `await`. Must return an iterator.

```
import asyncio

class AsyncValue:
    def __init__(self, value):
        self.value = value

    def __await__(self):
        yield # yield control to event loop once
        return self.value

async def main():
    result = await AsyncValue(42)
    print(result)

asyncio.run(main())    # 42
```

Q93. What is `__aiter__` and `__anext__`?

Async iteration protocol, used with `async for`.

```
import asyncio

class AsyncRange:
```



```

def __init__(self, n):
    self.n = n

def __aiter__(self):
    self.current = 0
    return self

async def __anext__(self):
    if self.current >= self.n:
        raise StopAsyncIteration
    await asyncio.sleep(0)
    val = self.current
    self.current += 1
    return val

async def main():
    async for x in AsyncRange(3):
        print(x)

asyncio.run(main())  # 0 1 2

```

Q94. What is `__aenter__` and `__aexit__`?

Async context manager protocol, used with `async with`.

```

import asyncio

class AsyncDB:
    async def __aenter__(self):
        print("Connecting...")
        await asyncio.sleep(0.01)
        return self

    async def __aexit__(self, *args):
        print("Disconnecting...")

async def main():
    async with AsyncDB() as db:
        print("Using db:", db)

asyncio.run(main())

```

Q95. What is `__or__` and how does it relate to `|` for dicts in Python 3.9+?

`dict.__or__` implements `d1 | d2` (merge). You can implement it for custom mappings.

```
class Config(dict):
    def __or__(self, other):
        merged = Config(self)
        merged.update(other)
        return merged

c1 = Config({"a": 1})
c2 = Config({"b": 2})
print(c1 | c2)    # {'a': 1, 'b': 2}
```

Q96. What is `__bytes__`?

Called by `bytes(obj)`.

```
class Packet:
    def __init__(self, data: list):
        self.data = data

    def __bytes__(self):
        return bytes(self.data)

p = Packet([72, 101, 108, 108, 111])
print(bytes(p))    # b'Hello'
```

Q97. What is `__complex__`, `__int__`, `__float__`?

Support type-coercion built-ins.

```
class Fraction:
    def __init__(self, num, den):
        self.num, self.den = num, den

    def __float__(self):
        return self.num / self.den

    def __int__(self):
        return self.num // self.den

f = Fraction(7, 2)
print(float(f))    # 3.5
print(int(f))      # 3
```

Q98. What is `__length_hint__`?

An optional optimisation hint for iterators. Called by `operator.length_hint()` to pre-allocate buffers without consuming the iterator.

```
class Range:
    def __init__(self, n):
        self.n = n
        self.i = 0

    def __iter__(self): return self

    def __next__(self):
        if self.i >= self.n: raise StopIteration
        v = self.i; self.i += 1; return v

    def __length_hint__(self):
        return max(0, self.n - self.i)
```

Q99. What is `__reversed__`?

Called by `reversed()`. If not defined, Python falls back to `__len__ + __getitem__`.

```
class Deck:
    def __init__(self, cards):
        self.cards = cards

    def __reversed__(self):
        return reversed(self.cards)

d = Deck(["A", "K", "Q"])
print(list(reversed(d))) # ['Q', 'K', 'A']
```

Q100. Putting it all together — implement a fully-featured `Vector2D` class.

```
import math
from functools import total_ordering

@total_ordering
class Vector2D:
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        object.__setattr__(self, "x", float(x))
```

```

        object.__setattr__(self, "y", float(y))

    # Prevent mutation
    def __setattr__(self, name, value):
        raise AttributeError("Vector2D is immutable")

    # Representation
    def __repr__(self):
        return f"Vector2D({self.x}, {self.y})"

    def __str__(self):
        return f"<{self.x}, {self.y}>"

    # Arithmetic
    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Vector2D(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar):
        return Vector2D(self.x * scalar, self.y * scalar)

    def __rmul__(self, scalar):
        return self.__mul__(scalar)

    def __truediv__(self, scalar):
        return Vector2D(self.x / scalar, self.y / scalar)

    def __neg__(self):
        return Vector2D(-self.x, -self.y)

    def __abs__(self):
        return math.hypot(self.x, self.y)

    # Equality and ordering (by magnitude)
    def __eq__(self, other):
        if not isinstance(other, Vector2D):
            return NotImplemented
        return (self.x, self.y) == (other.x, other.y)

    def __lt__(self, other):
        return abs(self) < abs(other)

    def __hash__(self):
        return hash((self.x, self.y))

```

```

# Iteration
def __iter__(self):
    yield self.x
    yield self.y

def __len__(self):
    return 2

def __getitem__(self, index):
    return (self.x, self.y)[index]

# Bool
def __bool__(self):
    return self.x != 0 or self.y != 0

# Format
def __format__(self, spec):
    if spec == "polar":
        return f"|{abs(self):.2f}| {math.degrees(math.atan2(self.y, self.x)):.1f}°"
    return str(self)

# Demo
v1 = Vector2D(3, 4)
v2 = Vector2D(1, 2)

print(repr(v1))           # Vector2D(3.0, 4.0)
print(v1 + v2)           # <4.0, 6.0>
print(abs(v1))           # 5.0
print(2 * v1)            # <6.0, 8.0>
print(f"{v1:polar}")     # |5.00| 53.1°
print(list(v1))          # [3.0, 4.0]
print(v1 > v2)           # True
print(hash(v1))          # stable int
print(bool(Vector2D(0,0))) # False

```

End of 100 Questions